

LEARNING PLAN – APPLY PROGRAMMING PRINCIPLES

1. INSTITUTIONAL DETAILS

ITEM	DETAILS
Unit of Competence	Apply Programming Principles
Unit Code	0613 451 05A
Name of Trainer	JAMES NDIRANGU
Competence Level	LEVEL 5
Institution	THE ABERDARE TECHNICAL COLLEGE

Number of Learners	3
Date of Preparation	06/08/2026
Skill or Job Task	applying computer programming skills, demonstrating structured programming skills and demonstrating object-oriented programming skills.

2. BENCHMARK / CRITERIA TO BE USED

1. Apply computer programming skills

1.1 Programming language types are identified as per the user requirements.

1.2 Programming paradigms are applied as per user requirements.

1.3 Program development life cycle is applied as per the work requirements.

1.4 Program design tools are applied as per the user requirements.

1.5 Program writing tools are identified as per the system requirements.

2. Demonstrate structured programming skills

2.1 Identifiers are declared as per program design specification.

2.2 Initialization of variables and constants is performed according to program design specifications.

2.3 User-defined data types are applied as per system requirements.

2.4 Computer program input is created as per program design.

2.5 Data control structures in a program are applied as per program design requirements.

2.6 Data structures in a program are applied as per program design specifications.

2.7 Computer program subroutines are created as per user needs.

2.8 Computer program output is coded as per user requirements.

2.9 Computer program debugging is performed as per work procedures.

2.10 Computer program is compiled as per system requirements.

3. Demonstrate object oriented programming skills

3.1 Objects and classes are implemented as per work procedures.

3.2 Objects methods are declared as per application requirements.

3.3 Namespaces are applied as per work procedures.

3.4 Data abstraction concepts are applied as per work procedures.

3.5 Object encapsulations are applied as per work procedures.

3.6 Class templates are implemented as per application requirements.

3.7 Class inheritance is implemented as per application requirements

. 3.8 Polymorphism is implemented as per application requirements.

LEARNING OUTCOME 1: APPLY COMPUTER PROGRAMMING SKILLS

WEEK 1

Week	Main Learnin g	Session Title /	Learning Outcome	Trainer Activities	Trainee Activities	Resource s	Assessme nt	Reflections and Dates
-------------	-------------------------------	----------------------------	-----------------------------	-------------------------------	-------------------------------	-----------------------	------------------------	----------------------------------

	Outcomes	Subtopics						
1	1. Apply computer programming skills	Session 1: Introduction to Programming Languages • Overview of programming languages• Programming language	By the end of the session, trainees should be able to:1. Define programming language.2. Identify programming language categories.3. Explain characteristics	1. Introduces programming languages and poses questions about their purpose.2. Explains programming language categories and asks trainees to identify them.3. Describes	1. Define programming language and answer questions about its purpose.2. Identify programming language categories.3. Explain the characteristics	Computer s, projector, programming language notes, sample programs	Oral questions , classification exercise	Date: _____ Reflection: _____

		categorie s• Procedur al language s• Object- oriented language s• Function al language s	ics of procedural, object- oriented and functional languages. 4. Differentia te programmi ng language categories. 5. Classify given programmi ng languages	procedural, object- oriented and functional languages and asks trainees to explain their characteristi cs.4. Compares language categories and asks trainees to differentiate them.5. Provides programmin	tics of procedural , object- oriented and functional languages. 4. Differenti ate the programm ing language categories. 5. Classify the given programm ing			
--	--	---	---	--	---	--	--	--

			according to their categories.	g language examples and guides trainees in classifying them.	languages according to their categories.			
1	1. Apply computer programming skills	Session 2: Selection of Programming Languages• Criteria for selecting programming	By the end of the session, trainees should be able to:1. Identify factors considered when selecting programming	1. Introduces language selection factors and poses questions on their importance. 2. Explains how user requirements influence	1. Identify factors considered when selecting programming languages and answer questions. 2. Explain	Computers, projector, programming language comparison charts, case studies	Language selection case study	Date: _____ Reflection: _____

		language s• User requirem ents• System requirem ents• Project requirem ents• Develope r considera tions	languages. 2. Explain how user requiremen ts influence language selection.3. Analyse programmi ng requiremen ts.4. Compare programmi ng languages for a given project.5. Select an	selection and asks trainees to identify examples.3. Provides project requirement s and guides trainees in analysing them.4. Provides language alternatives and guides trainees in comparing them.5.	how user requireme nts influence language selection.3 . Analyse the project requireme nts provided.4 . Compare the programm ing language alternative s.5. Select			
--	--	--	--	--	--	--	--	--

			appropriate programming language for a given requirement.	Presents project scenarios and asks trainees to select suitable languages.	the most appropriate programming language for each scenario.			
--	--	--	---	--	--	--	--	--

WEEK 2

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
2	1. Apply computer program	Session 3: Application Programm	By the end of the session, trainees	1. Introduces programming	1. Define programming paradigms	Computers, projector ,	Paradigm comparison exercise,	Date: _____ Reflection: _____

	ming skills	ing Paradigms • Common programming paradigms • Functional • Procedural • Object-oriented • Imperative • Declarative	should be able to:1. Define programming paradigms. 2. Identify common programming paradigms. 3. Explain the characteristics of each paradigm. 4. Differentiate imperative,	paradigms and poses questions about their purpose.2. Identifies common paradigms and asks trainees to identify examples.3. Explains the characteristics of each paradigm and asks trainees to	and answer questions about their purpose.2. Identify common programming paradigms and their examples. 3. Explain the characteristics of each paradigm. 4.	paradigm diagram s, sample programs	oral questions	
--	--------------------	--	---	--	--	-------------------------------------	----------------	--

			declarative, procedural, functional and object-oriented paradigms. 5. Analyse applications of different programming paradigms.	describe them.4. Compares paradigms and guides trainees in differentiating them.5. Provides application scenarios and guides trainees in analysing suitable paradigms.	Differentiate the programming paradigms. 5. Analyse the application scenarios and identify suitable paradigms.			
2	1. Apply computer	Session 4: Selection of	By the end of the session,	1. Explains factors affecting	1. Identify factors influencing	Computers, case studies,	Paradigm selection	Date: _____ Reflection: _____

	program ming skills	Programm ing Paradigms • Choosing appropriate paradigms • Project requirements • User requirements • Project complexity	trainees should be able to: 1. Identify factors influencing paradigm selection. 2. Explain the relationship between project needs and paradigms. 3. Analyse project requirements. 4. Evaluate	paradigm selection and poses questions about them. 2. Demonstrates how project needs influence paradigm selection and asks trainees to identify relationship s.3. Provides	g paradigm selection and answer questions. 2. Explain how project needs influence paradigm selection. 3. Analyse the project requirements provided. 4. Evaluate	projector , paradigm charts	case study	
--	---	--	---	---	---	------------------------------------	-------------------	--

			alternative programming paradigms. 5. Select an appropriate paradigm for a given project.	project requirements and guides trainees in analysing them.4. Provides alternative paradigms and guides trainees in evaluating them.5. Provides project scenarios and asks trainees to	alternative programming paradigms. 5. Select an appropriate paradigm for the given project.			
--	--	--	--	--	--	--	--	--

				select appropriate paradigms.				
--	--	--	--	-------------------------------------	--	--	--	--

WEEK 3

Week	Main Learnin g Outcom e	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resource s	Assessm ent	Reflections and Dates
-------------	--	--	-----------------------------	-------------------------------	-------------------------------	-----------------------	------------------------	----------------------------------

3	1. Apply computer programming skills	Session 5: Program Development Life Cycle• Stages of program development life cycle• Requirements analysis• Program design• Coding• Testing• Implementation• Maintenance	By the end of the session, trainees should be able to:1. Define program development life cycle.2. Identify stages of the program development life cycle.3. Explain	1. Introduce the program development life cycle and poses questions about its purpose.2. Identifies the stages and asks trainees to identify them.3. Explains the	1. Define the program development life cycle and answer questions about its purpose.2. Identify the stages of the life cycle.3. Explain the purpose of each stage.4.	Computers, projector, PDLC charts, notes	Stage sequencing exercise, oral questions	Date: _____ Reflection: _____
---	--------------------------------------	--	---	--	--	--	---	----------------------------------

			the purpose of each stage.4. Sequence the stages correctly.5. Analyse the importance of following the program development life cycle.	purpose of each stage and asks trainees to describe it.4. Demonstrates the correct sequence and guides trainees in arranging the stages.5. Provides development scenarios	Sequence the stages correctly. 5. Analyse the importance of following the life cycle.			
--	--	--	--	--	---	--	--	--

				and guides trainees in analysing the importance of the life cycle.				
3	1. Apply computer programming skills	Session 6: Adapting the Program Development Life Cycle • Best practices• Adapting the life cycle to work	By the end of the session, trainees should be able to:1. Identify best practices in program development.2.	1. Introduce programming best practices and poses questions on their importance.2. Explains	1. Identify programming best practices and answer questions about their importance.2.	Compute rs, project scenarios, development templates	Project development exercise	Date: _____Reflection: _____

		requirement s• Project constraints• User requirement s	Explain factors affecting adaptation of the life cycle.3. Analyse work requiremen ts.4. Evaluate alternative developme nt approaches. 5. Develop an appropriate program	factors affecting life-cycle adaptation and asks trainees to identify them.3. Provides work requireme nts and guides trainees in analysing them.4. Provides developme nt	Explain factors affecting life-cycle adaptation .3. Analyse the work requireme nts provided. 4. Evaluate alternative developm ent approache s.5. Develop			
--	--	---	---	--	--	--	--	--

			developme nt approach for a given project.	approaches and guides trainees in evaluating them.5. Provides a project scenario and guides trainees in developing an appropriat e approach.	an appropriat e developm ent approach for the given project.			
--	--	--	---	---	--	--	--	--

WEEK 4

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
4	1. Apply computer programming skills	Session 7: Program Design Tools • Flowcharts • Decision tables• Decision trees• Pseudocode• Algorithms	By the end of the session, trainees should be able to:1. Identify common program design tools.2. Explain the purpose of	1. Introduces program design tools and poses questions about their uses.2. Explains each design tool and asks trainees to	1. Identify program design tools and answer questions about their uses.2. Explain the purpose of each design tool.3.	Computers, projector, flowchart software, draw.io, worksheets	Algorithm, pseudocode and flowchart practical	Date: _____ Reflect ion: _____

			flowcharts, decision tables, decision trees, pseudocode and algorithms. 3. Differentiate program design tools.4. Construct algorithms and pseudocode.5.	state its purpose.3. Compares design tools and guides trainees in differentiating them.4. Demonstrates algorithm and pseudocode development and guides trainees in constructing	Differentiate the design tools.4. Construct algorithms and pseudocode for given problems. 5. Develop flowcharts for the programming problems provided.		
--	--	--	--	--	--	--	--

			Develop flowcharts for given programming problems.	g them.5. Demonstrates flowchart development and guides trainees in developing flowcharts.				
4	1. Apply computer programming skills	Session 8: Selection of Program Design Tools• User requirements•	By the end of the session, trainees should be able to:1. Identify factors affecting	1. Explains factors affecting design-tool selection and poses questions about them.2.	1. Identify factors affecting design-tool selection and answer questions.	Computers, case studies, draw.io, design worksheets	Design-tool selection exercise	Date: _____ Reflect ion: _____

		Project complexity • Selecting appropriate design tools	selection of design tools.2. Explain the relationship between requirements and design tools.3. Analyse programming problems.4 . Evaluate alternative design tools.5.	Demonstrates how requirements influence tool selection and asks trainees to identify examples.3 . Provides programming problems and guides trainees in analysing them.4. Provides	2. Explain how requirements influence design-tool selection.3 . Analyse the programming problems provided.4 . Evaluate alternative design tools.5. Select			
--	--	--	--	--	--	--	--	--

			Select appropriate design tools for given programm ing problems.	alternative tools and guides trainees in evaluating them.5. Provides project scenarios and asks trainees to select suitable tools.	appropriat e design tools for the given problems.			
--	--	--	--	--	---	--	--	--

WEEK 5

Week	Main Learnin g	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resourc es	Assessmen t	Reflections and Dates
------	----------------------	---------------------------------	---------------------	-----------------------	-----------------------	---------------	----------------	--------------------------

	Outcome							
5	1. Apply computer programming skills	Session 9: Program Writing Tools and IDEs• Text editors• Compilers• Linkers• Debuggers • Integrated Development Environment Environments	By the end of the session, trainees should be able to:1. Identify common program writing tools.2. Explain the functions of text editors, compilers,	1. Introduce s program writing tools and poses questions about their purposes.2 . Explains text editors, compilers, linkers and debuggers and asks trainees to	1. Identify program writing tools and answer questions about their purposes.2. Explain the functions of text editors, compilers, linkers and debuggers. 3. Identify	Computers, C/C++ compiler, IDE, text editor, projector	Tool identification, practical exercise	Date: _____ Reflection: _____

			linkers and debuggers. 3. Describe the functions of an IDE.4. Differentiate program writing tools.5. Demonstrate the use of basic program writing tools.	identify their functions.3 . Demonstrates IDE features and asks trainees to identify them.4. Compares program writing tools and guides trainees in differentiating	the main features of an IDE.4. Differentiate program writing tools according to their functions.5. Demonstrate basic use of the program writing tools.			
--	--	--	---	---	---	--	--	--

				them.5. Demonstrates basic tool usage and guides trainees in practising it.				
5	1. Apply computer programming skills	Session 10: Evaluation and Selection of Programming Tools• System requirements	By the end of the session, trainees should be able to:1. Identify factors used to evaluate programmi	1. Explains tool evaluation criteria and poses questions on their importance .2. Demonstr	1. Identify programming-tool evaluation criteria and answer questions.2 . Explain how system requirements influence	Computers, IDEs, compiler, linker, debugger	CAT 1: Programming Language s, Paradigm s, PDLC and Design Tools	Date: _____ Reflection: _____

		ts• Developer preference s• Tool compatibility• IDE selection	ng tools.2. Explain how system requirements influence tool selection.3. Analyse developer and project requirements.4. Evaluate available programming tools.5. Select appropriate programming	ates the relationship between system requirements and tools and asks trainees to identify suitable examples. 3. Provides project requirements and guides trainees in	tool selection.3. Analyse the project and developer requirements.4. Evaluate the available programming tools.5. Select suitable programming tools for the project.			
--	--	--	--	---	--	--	--	--

			ng tools for a project.	analysing them.4. Provides different programmi ng tools and guides trainees in evaluating them.5. Provides project scenarios and asks trainees to select suitable tools.				
--	--	--	--------------------------------	---	--	--	--	--

LEARNING OUTCOME 2: DEMONSTRATE STRUCTURED PROGRAMMING SKILLS

WEEK 6

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
6	2. Demonstrate structured programming skills	Session 11: C Identifiers, Variables and Constants • Declaration of identifier	By the end of the session, trainees should be able to:1. Define identifiers, variables and constants.2.	1. Introduce C identifiers, variables and constants and poses questions about their purposes.2.	1. Define identifiers, variables and constants and answer questions about their purposes.2. Identify	Computers, C compiler/IDE, projector, C programming notes	Identifier and initialization practical	Date: _____ Reflection: _____

		• Naming conventions • Best practices • Variables • Constants • Initialization	Identify valid and invalid C identifiers.3 . Apply naming conventions when declaring identifiers.4 . Initialize variables and constants correctly.5. Evaluate declarations against program	Demonstrates valid and invalid identifiers and asks trainees to identify them.3. Explains naming convention s and guides trainees in applying them.4. Demonstrates variable	valid and invalid C identifiers. 3. Apply correct naming convention s when declaring identifiers.4. Initialize variables and constants correctly.5 . Evaluate declaration s against			
--	--	--	--	---	--	--	--	--

			design specificatio ns.	and constant initializatio n and guides trainees in practising it.5. Provides declaration s and asks trainees to evaluate them against specificati ons.	the given specificati ons.			
--	--	--	-------------------------------	--	----------------------------------	--	--	--

6	2. Demonstrate structured programming skills	Session 12: User-Defined Data Types in C • Structure • Classes • Arrays • Functions • Criteria for selecting data types	By the end of the session, trainees should be able to:1. Identify common data types used in C programming.2. Explain structures, arrays and functions.3. Differentiate the appropriate	1. Introduce s data types and poses questions about their uses.2. Explains structures, arrays and functions and asks trainees to describe them.3. Compares data types and guides	1. Identify common C data types and answer questions about their uses.2. Explain structures, arrays and functions.3. Differentiate the uses of different data types.4.	Computers , C compiler/IDE, sample programs	Data-type selection practical	Date: _____Reflection: _____
---	--	---	---	--	--	---	-------------------------------	------------------------------

			uses of data types.4. Apply suitable data types to programming problems.5. Select appropriate data types based on system requirements.	trainees in differentiating their uses.4. Demonstrates data-type application and guides trainees in applying suitable types.5. Provides system requirements and asks trainees to select	Apply suitable data types to programming problems.5 . Select appropriate data types based on the given requirements.			
--	--	--	--	---	---	--	--	--

				appropriate types.				
--	--	--	--	--------------------	--	--	--	--

WEEK 7

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
7	2. Demonstrate structured programming skills	Session 13: Creating C Program Input and Output • Program input•	By the end of the session, trainees should be able to:1. Explain program input and output.2.	1. Explains program input and output and poses questions about their purposes.2. Demonstr	1. Explain program input and output and answer questions.2. Identify appropriate input	Computers , C compiler/IDE, sample programs	Input/output programming practical	Date: _____ Reflection: _____

		Input statement s• Program output• Output statement s	Identify appropriate input and output statements .3. Apply input statements in C programs. 4. Apply output statements in C programs. 5. Create C programs that accept	ates input and output statements and asks trainees to identify them.3. Demonstrates input coding and guides trainees in applying input statements. 4. Demonstrates output coding and	and output statements .3. Apply input statements in C programs. 4. Apply output statements in C programs. 5. Create C programs that accept input and produce the			
--	--	---	--	--	--	--	--	--

			input and produce output.	guides trainees in applying output statements. 5. Provides programming tasks and guides trainees in creating complete input/output programs.	required output.			
7	2. Demonstrate structure	Session 14: C Control Structures	By the end of the session, trainees	1. Introduces control structures	1. Identify control structures	Computers, C compiler/IDE,	Control-structure practical	Date: _____ Reflection: _____

	red progra mming skills	es• Sequence • Selection • Loops• Best practices in control structure s	should be able to:1. Identify types of control structures. 2. Explain sequence, selection and iteration.3. Differenti ate the control structures. 4. Apply appropriat e control structures	and poses identificati on questions.2 . Explains sequence, selection and loops and asks trainees to describe them.3. Compares control structures and guides trainees in differentiat ing them.4.	and answer identificati on questions. 2. Explain sequence, selection and loops.3. Differenti ate the control structures. 4. Apply appropriat e control structures in C	flowcharts, sample programs		
--	--	---	--	---	--	-----------------------------------	--	--

			in C programs. 5. Evaluate control structures against design requirements.	Demonstrates control structures and guides trainees in applying them.5. Provides program designs and asks trainees to evaluate the selected control structures.	programs. 5. Evaluate the control structures against the design requirements.			
--	--	--	---	--	--	--	--	--

WEEK 8

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
8	2. Demonstrate structured programming skills	Session 15: Data Structures in C• Arrays• Queues• Stacks• Linked lists• Selection of appropriate data structures	By the end of the session, trainees should be able to:1. Identify common data structures.2. Explain arrays, queues, stacks and	1. Introduce s data structures and poses questions about their purpose.2. Explains arrays, queues, stacks and linked lists and asks	1. Identify common data structures and answer questions about their purpose.2. Explain arrays, queues, stacks and linked	Computers, C compiler/IDE, data structure diagrams, sample programs	Data structure practical	Date: _____ Reflection: _____

			linked lists.3. Differentiate the data structures. 4. Apply data structures in C programs. 5. Select appropriate data structures based on design specifications.	trainees to describe them.3. Compares data structures and guides trainees in differentiating them.4. Demonstrates data structure implementation and guides trainees in applying	lists.3. Differentiate the data structures. 4. Apply data structures in C programs. 5. Select appropriate data structures for the given specifications.			
--	--	--	---	---	---	--	--	--

				them.5. Provides design specificati ons and asks trainees to select suitable structures.				
8	2. Demonstrate structured programming skills	Session 16: C Program Subroutines and Functions • Benefits of	By the end of the session, trainees should be able to:1. Explain the purpose of	1. Explains subroutine s and poses questions about their benefits.2. Demonstr	1. Explain the purpose of subroutine s and answer questions about their benefits.2.	Computer s, C compiler/ IDE, sample programs	Functions/subr outines practical	Date: _____ Refle ction: _____

		subroutines• Function s and subprograms• Designing subroutines• User needs	subroutine s.2. Identify benefits of using subroutine s.3. Differentiate functions and subprograms.4. Design subroutine s to meet user needs.5. Create C	ates benefits of modular programming and asks trainees to identify them.3. Explains functions and subprograms and guides trainees in differentiating them.4.	Identify benefits of using subroutine s.3. Differentiate functions and subprograms.4. Design subroutine s to meet the stated user needs.5. Create C functions			
--	--	--	--	---	---	--	--	--

			functions and subroutine s.	Demonstrates subroutine design and guides trainees in designing subroutine s.5. Demonstrates function creation and guides trainees in creating functions.	and subroutine s.			
--	--	--	--------------------------------------	---	-------------------------	--	--	--

WEEK 9

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
9	2. Demonstrate structured programming skills	Session 17: C Program Debugging • Common debugging techniques • Debugging tools• Systematic debugging procedures	By the end of the session, trainees should be able to:1. Define program debugging.2. Identify common programming errors.3. Explain	1. Explains debugging and poses questions about its importance .2. Demonstrates common programming errors and asks	1. Define debugging and answer questions about its importance .2. Identify errors in the sample programs.3. Explain and apply	Computers , C compiler/IDE, debugger, faulty programs	Debugging practical	Date: _____ Reflection: _____

			common debugging techniques. 4. Apply systematic debugging procedures 5. Debug C programs using appropriate tools.	trainees to identify them. 3. Demonstrates debugging techniques and guides trainees in applying them. 4. Demonstrates systematic debugging procedures and guides trainees through the	common debugging techniques. 4. Follow systematic debugging procedures 5. Debug the faulty C programs using appropriate tools.			
--	--	--	---	--	---	--	--	--

				process.5. Provides faulty C programs and guides trainees in debugging them.				
9	2. Demonstrate structured programming skills	Session 18: C Program Compilation• Compilation process• Compiler• Linker• Executable	By the end of the session, trainees should be able to:1. Explain the compilation process.2.	1. Explains the compilation process and poses questions about its stages.2. Demonstrates the	1. Explain the compilation process and answer questions about its stages.2. Identify the stages	Computers , C compiler, IDE, sample programs	Compilation practical	Date: _____ Reflection: _____

		e program• System requireme nts	Identify the stages involved in compilatio n.3. Describe the roles of the compiler and linker.4. Compile C programs successfull y.5. Evaluate compiled programs against	compilatio n stages and asks trainees to identify them.3. Explains compiler and linker functions and asks trainees to describe them.4. Demonstr ates C program compilatio n and	involved in compilatio n.3. Describe the roles of the compiler and linker.4. Compile C programs successfull y.5. Evaluate the compiled programs against the given			
--	--	--	---	--	--	--	--	--

			system requiremen ts.	guides trainees in compiling programs.5 . Provides system requiremen ts and asks trainees to evaluate their compiled programs.	requiremen ts.			
--	--	--	-----------------------------	---	-------------------	--	--	--

WEEK 10

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
10	2. Demonstrate structured programming skills	Session 19: Integrated Structured C Programming • Identifiers• Variables and constants• Data types• Input and output• Control	By the end of the session, trainees should be able to:1. Analyse a programming problem.2. Design a structured C program.3	1. Provides a programming problem and guides trainees in analysing it.2. Demonstrates a structured program design and	1. Analyse the programming problem provided.2. Design a structured C program solution.3. Apply	Computers , C compiler/IDE, programming task, debugger	CAT 2 + Practical Assessment 1: Structured C Programming	Date: _____ Reflection: _____

		structures• Data structures• Functions• Debugging• Compilation	. Apply appropriate variables, data types and control structures. 4. Create functions and data structures. 5. Debug the developed program.6 . Compile and evaluate	guides trainees in designing solutions.3 . Guides selection and application of variables, data types and control structures. 4. Demonstr ates function and data	appropriate variables, data types and control structures. 4. Create functions and data structures in the program.5 . Debug the developed program and correct			
--	--	---	---	---	---	--	--	--

			the completed C program.	structure integration and guides trainees in creating them.5. Provides debugging support and guides trainees in correcting errors.6. Supervise s compilation and asks trainees to evaluate	errors.6. Compile and evaluate the completed C program.			
--	--	--	--------------------------	---	--	--	--	--

				their completed programs.				
--	--	--	--	---------------------------	--	--	--	--

LEARNING OUTCOME 3: DEMONSTRATE OBJECT-ORIENTED PROGRAMMING SKILLS

WEEK 11

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
11	3. Demonstrate object-oriented	Session 20: Objects and Classes in C++•	By the end of the session, trainees should be able to:1.	1. Introduce s OOP and poses questions about its	1. Define OOP and answer questions about its purpose.2.	Computers , C++ IDE/compiler, projector, OOP notes	Class and object creation practical	Date: _____Reflection: _____

	programming skills	Introduction to OOP• Objects• Classes• Work procedure s	Define object-oriented programming.g.2. Identify objects and classes.3. Explain the relationship between objects and classes.4. Differentiate objects from classes.5. Create basic C++	purpose.2. Explains objects and classes and asks trainees to identify them.3. Demonstrates the relationship between classes and objects and asks trainees to explain it.4. Compares	Identify objects and classes from the given examples.3. Explain the relationship between objects and classes.4. Differentiate objects from			
--	---------------------------	---	---	---	---	--	--	--

			classes and objects.	objects and classes and guides trainees in differentiating them.5. Demonstrates class and object creation and guides trainees in creating them.	classes.5. Create basic C++ classes and objects.			
11	3. Demonstrate object-orientation	Session 21: Object Methods in C++•	By the end of the session, trainees should be	1. Introduce s object methods and poses	1. Define object methods and answer	Computers , C++ IDE/compiler, sample programs	Method creation practical	Date: _____ Reflection: _____

	d progra mming skills	Object methods• Method declaratio n• Applicatio n requireme nts• Method naming and functionali ty	able to:1. Define object methods.2. Explain the purpose of methods.3. Declare object methods in C++.4. Apply appropriate method naming conventions .5. Create methods that fulfil	questions about their purpose.2. Explains method functionali ty and asks trainees to identify suitable uses.3. Demonstr ates method declaration and guides trainees in declaring methods.4.	questions about their purpose.2. Explain method functional ity and identify suitable uses.3. Declare object methods in C++.4. Apply appropriat e method naming			
--	--	--	--	--	--	--	--	--

			application requirements.	Demonstrates method naming practices and asks trainees to apply them.5. Provides application requirements and guides trainees in creating suitable methods.	practices. 5. Create methods that fulfil the given application requirements.			
--	--	--	---------------------------	--	--	--	--	--

WEEK 12

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates
12	3. Demonstrate object-oriented programming skills	Session 22: Namespaces and Data Abstraction in C++ Role of namespaces • Implementing namespaces •	By the end of the session, trainees should be able to:1. Explain the purpose of namespaces .2. Implement namespaces in C++ programs.3.	1. Explains namespaces and poses questions about their purpose.2. Demonstrates namespace implementation and guides trainees in implementi	1. Explain the purpose of namespace s and answer questions about their use.2. Implement namespace s in C++ programs.	Computers, C++ IDE/compiler, OOP diagrams, sample programs	Namespace and abstraction practical	Date: _____ Reflection: _____

		Definition of data abstraction - Importance of data abstraction - Implementing data abstraction	Define data abstraction. 4. Explain the importance of data abstraction. 5. Implement data abstraction in C++ programs.	ng namespaces .3. Explains data abstraction and asks trainees to define it.4. Discusses the importance of abstraction and asks trainees to explain its benefits.5. Demonstra	3. Define data abstraction .4. Explain the importance and benefits of data abstraction .5. Implement data abstraction in C++ programs.			
--	--	--	---	--	---	--	--	--

				tes abstraction implementa tion and guides trainees in implementi ng it.				
12	3. Demons trate object- oriente d progra mming skills	Session 23: Object Encapsula tion in C++• Definition of encapsulati on• Importanc e of	By the end of the session, trainees should be able to:1. Define object encapsulati on.2. Explain the	1. Defines encapsulati on and poses questions about its meaning.2. Explains the importance of	1. Define encapsulat ion and answer questions about its meaning.2 . Explain the importanc e of	Computer s, C++ IDE/comp iler, sample programs	Encapsula tion practical	Date: _____Refle ction: _____

		encapsulati on• Implement ing encapsulati on	importance of encapsulati on.3. Identify encapsulati on in C++ programs.4. Apply access control mechanism s.5. Implement encapsulati on in C++ classes.	encapsulati on and asks trainees to identify its benefits.3. Demonstra tes encapsulate d classes and asks trainees to identify encapsulati on.4. Demonstra tes access control and guides trainees in	encapsulat ion and identify its benefits.3. Identify encapsulat ion in sample C++ programs. 4. Apply appropriat e access control mechanis ms.5. Implemen t			
--	--	---	--	---	---	--	--	--

				applying it.5. Demonstrates encapsulation on implementation and guides trainees in implementing it.	encapsulation in C++ classes.			
--	--	--	--	---	-------------------------------------	--	--	--

WEEK 13

Week	Main Learning Outcome	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resources	Assessment	Reflections and Dates

13	3. Demonstrate object-oriented programming skills	Session 24: Class Templates in C++ • Class templates• Generic programming• Template implementation	By the end of the session, trainees should be able to:1. Define class templates.2. Explain the purpose of class templates.3. Identify applications of	1. Introduces class templates and poses questions about their purpose.2. Explains generic programming and asks trainees to explain its benefits.3. Demonstrates applications of	1. Define class templates and answer questions about their purpose.2. Explain generic programming and state its benefits.3. Identify applications of templates	Computers , C++ IDE/compiler, template examples	Class-template practical	Date: _____Reflection: _____
----	--	---	--	--	---	---	--------------------------	------------------------------

			templates. 4. Apply class templates in C++ programs. 5. Create a C++ program using class templates.	templates and asks trainees to identify them.4. Demonstrates template implementation and guides trainees in applying templates.5. Provides a programming task and guides trainees in creating a	from the examples provided. 4. Apply class templates in C++ programs. 5. Create a C++ program using a class template.			
--	--	--	--	---	--	--	--	--

				template-based program.				
13	3. Demonstrate object-oriented programming skills	Session 25: Class Inheritance in C++• Base classes• Derived classes• Inheritance relationship s• Types of inheritance	By the end of the session, trainees should be able to:1. Define inheritance e.2. Explain the importance of inheritance e.3. Identify	1. Introduces inheritance and poses questions about its purpose.2. Explains the importance of inheritance and asks trainees to explain its benefits.3.	1. Define inheritance and answer questions about its purpose.2. Explain the importance of inheritance e and state its benefits.3. Identify	Computers , C++ IDE/compiler, inheritance diagrams, sample programs	Inheritance practical	Date: _____ Reflection: _____

			base and derived classes.4. Differentiate types of inheritance.5. Implement inheritance relationships in C++.	Demonstrates base and derived classes and asks trainees to identify them.4. Compares types of inheritance and guides trainees in differentiating them.5. Demonstrates inheritance implementation	base and derived classes in the examples provided. 4. Differentiate the types of inheritance.5. Implement inheritance relationships in C++.			
--	--	--	---	---	---	--	--	--

				tion and guides trainees in implementi ng relationship s.				
--	--	--	--	--	--	--	--	--

WEEK 14

Week	Main Learni ng	Session Title / Subtopics	Learning Outcome	Trainer Activities	Trainee Activities	Resource s	Assessme nt	Reflections and Dates
-------------	-------------------------------	--	-----------------------------	-------------------------------	-------------------------------	-----------------------	------------------------	----------------------------------

	Outcome							
14	3. Demonstrate object-oriented programming skills	Session 26: Polymorphism in C++• Definition of polymorphism• Importance of polymorphism• Implementing polymorphism	By the end of the session, trainees should be able to:1. Define polymorphism.2. Explain the importance of polymorphism.3. Identify polymorphism in C++	1. Defines polymorphism and poses questions about its meaning.2. Explains the importance of polymorphism and asks trainees to state its benefits.3.	1. Define polymorphism and answer questions about its meaning.2. Explain the importance of polymorphism and state its benefits.3. Identify polymorphism in the	Computer s, C++ IDE/compiler, sample programs, OOP diagrams	Polymorphism practical	Date: _____ Reflection: _____

			programs.4. Differentiate forms of polymorphism.5. Implement polymorphism in C++ programs.	Demonstrates polymorphic programs and asks trainees to identify polymorphism.4. Compares forms of polymorphism and guides trainees in differentiating them.5. Demonstrates	sample C++ programs.4. Differentiate the forms of polymorphism.5. Implement polymorphism in C++ programs.			
--	--	--	---	--	--	--	--	--

				polymorphism implementation and guides trainees in implementing it.				
14	3. Demonstrate object-oriented programming skills	Session 27: Integrated Object-Oriented C++ Programming• Classes and objects•	By the end of the session, trainees should be able to:1. Analyse an object-oriented programming	1. Provides an OOP programming problem and guides trainees in analysing it.2. Demonstrates class and object	1. Analyse the OOP programming problem provided.2. Design appropriate classes and objects.3. Apply methods,	Computer s, C++ IDE/compiler, project specifications, sample programs	CAT 3 + Practical Assessment 2: Object-Oriented Programming	Date: _____ Reflection: _____

		Methods• Namespaces• Abstraction• Encapsulation• Class templates• Inheritance • Polymorphism	problem.2. Design appropriate classes and objects.3. Apply methods, namespaces , abstraction and encapsulation.4. Implement class templates and inheritance. 5. Implement	design and guides trainees in designing classes.3. Guides trainees in applying methods, namespaces , abstraction and encapsulation.4. Demonstrates templates and inheritance	namespaces , abstraction and encapsulation.4. Implement class templates and inheritance. 5. Implement polymorphism in the application. 6. Evaluate the completed C++			
--	--	---	--	---	---	--	--	--

			polymorphism.6. Evaluate the completed C++ application against user requirements.	and guides trainees in implementing them.5. Demonstrates polymorphism and guides trainees in implementing it.6. Reviews completed applications and asks trainees to evaluate them	application against user requirements.			
--	--	--	--	---	---	--	--	--

				against requirement s.				
--	--	--	--	------------------------------	--	--	--	--